



Universidade Estadual Paulista “Júlio de Mesquita Filho” (UNESP)
Faculdade de Ciências – Bauru
Departamento de Computação

RANK-IA

Formação otimizada de equipes com Inteligência Artificial

Engenharia de Software II

Fernando Hiroshi Murusaki – RA: 241025851

Igor dos Reis Gomes – RA: 241025265

Matheus Santos Magro – RA: 231025335

Murilo Tomaz Gonzaga – RA: 241024684

Sumário

Como utilizar este modelo	1
1 Introdução e Objetivos	2
1.1 Contexto do software	2
1.2 Escopo	3
1.3 Visão geral dos requisitos	4
1.4 Regras de negócio	5
1.5 Objetivos de qualidade	6
2 Arquitetura do Sistema	8
2.1 Contexto e fronteiras do sistema	8
2.2 Estilo ou padrão arquitetural	8
2.3 Contêineres e componentes	8
2.4 Decisões arquiteturais relevantes	8
2.5 Restrições de arquitetura	9
3 Projeto de Componentes	10
3.1 Visão geral dos componentes	10
3.2 Detalhamento dos componentes principais	10
3.3 Princípios de projeto	10
3.4 Padrões de projeto	11
4 Projeto de Interface do Usuário	12
5 Especificação e Estratégia de Testes	13
5.1 Planejamento e níveis de teste	13
5.2 Testes de unidade	14
5.2.1 <i>Test-Driven Development</i>	14
5.3 Testes de integração	15
5.4 Teste de validação e aceitação	15
5.5 Teste de sistema	16
5.6 Critérios de conclusão e cobertura	17
5.7 Depuração	17
5.8 Automação e ferramentas	17
6 Gestão de Configuração e Manutenção	19
6.1 Repositório e itens de configuração	19
6.1.1 Documentação em três camadas	20
6.1.2 O que fica fora do repositório	21
6.1.3 Organização de <i>branches</i> e versões	21
6.2 Gestão de dependências e alterações	22

6.3	Rastreabilidade	22
6.4	<i>Build</i> , integração e entrega	24
6.4.1	Construção local	24
6.4.2	Integração contínua	24
6.4.3	Ambientes e entrega	25

Como utilizar este modelo

Este documento é um roteiro para apoiar a documentação do projeto final. Os textos marcados como *Orientação* explicam o que se espera em cada parte e podem ser ocultados alterando-se, no início do arquivo, `\orientacoestrue` para `\orientacoesfalse`.

O documento deve registrar as principais decisões do projeto, e não apenas reunir diagramas. Sempre que uma escolha relevante for apresentada, expliquem brevemente a motivação e suas consequências. Diagramas, tabelas e protótipos devem ser mencionados e explicados no texto.

Não é necessário documentar todas as classes, telas ou detalhes internos do sistema. Em projetos maiores, priorizem os elementos mais relevantes, complexos ou representativos.

Capítulo 1

Introdução e Objetivos

***Orientação:** Apresentem brevemente o sistema e expliquem o que será documentado neste capítulo. O objetivo é permitir que um leitor compreenda o problema e o escopo antes de examinar as decisões de projeto.*

O RANK-IA é uma aplicação web corporativa que apoia a formação de equipes de trabalho dentro de uma organização. A partir das habilidades, competências e lacunas de formação (*gaps*) de cada colaborador, um modelo de Inteligência Artificial sugere composições de equipe e associa a cada uma um **Coefficiente de Aproveitamento**, que estima o desempenho esperado do grupo. Como a formação de equipes envolve fatores humanos nem sempre mensuráveis, o gestor pode ajustar as sugestões manualmente, e esses ajustes retroalimentam o modelo: automação assistida por supervisão humana, não decisão automática.

Este capítulo delimita o problema, o escopo, os requisitos mais relevantes e os atributos de qualidade que orientam as decisões dos capítulos seguintes; a especificação detalhada de requisitos, os casos de uso e os diagramas de análise, produzidos na etapa anterior do projeto, são aqui apenas referenciados, sem repetição integral.

1.1 Contexto do software

***Orientação:** Contextualizem o problema do ponto de vista dos usuários e da atividade que o software deve apoiar, automatizar ou melhorar. Identifiquem os principais usuários ou partes interessadas quando isso for relevante.*

Na maioria das organizações, a montagem de equipes é uma atividade manual e pouco sistematizada: o profissional de RH ou o gestor precisa cruzar currículos, planilhas de competências, histórico de projetos anteriores e conhecimento tácito sobre cada pessoa. Esse processo apresenta três limitações recorrentes:

- **Custo de tempo.** A informação necessária está dispersa em documentos de formatos distintos, e o esforço de consolidá-la cresce rapidamente com o número de colaboradores considerados.
- **Subjetividade.** Sem um critério explícito, a escolha depende fortemente da familiaridade do gestor com os candidatos, o que tende a favorecer sempre os mesmos perfis e a deixar competências pouco visíveis fora das composições.
- **Ausência de rastreabilidade.** Não fica registrado por que uma determinada equipe foi formada, o que dificulta revisar a decisão, comparar alternativas e aprender com os resultados obtidos.

O RANK-IA atua exatamente sobre essa atividade: centraliza os dados de competências, torna explícito o critério de composição pelo Coeficiente de Aproveitamento e mantém o histórico das equipes geradas e dos ajustes manuais. Um efeito secundário é que as lacunas

identificadas passam a alimentar sugestões de treinamento, ligando a formação de equipes ao desenvolvimento profissional dos colaboradores.

As principais partes interessadas do sistema estão descritas na Tabela 1.1.

Tabela 1.1: Partes interessadas do RANK-IA.

Parte interessada	Tipo	Interesse no sistema
Gestor / RH	Usuário primário	Cadastrar colaboradores, parametrizar os critérios da IA, solicitar e validar equipes, acompanhar relatórios. Possui acesso total ao sistema.
Colaborador	Usuário secundário	Consultar as próprias competências e <i>gaps</i> , acompanhar treinamentos sugeridos e visualizar as equipes das quais participa. Possui acesso restrito aos seus próprios dados.
Organização	Patrocinador	Reduzir o tempo de montagem de equipes, aumentar a produtividade dos times e manter a operação em conformidade com a LGPD.
Equipe de desenvolvimento	Executor	Evoluir o modelo de IA e os módulos do sistema sem regressões, com apoio de testes automatizados.

1.2 Escopo

Orientação: *Delimitem o que faz parte do sistema e, quando necessário, o que está explicitamente fora do escopo. Evitem transformar esta seção em uma especificação detalhada de requisitos.*

O escopo do RANK-IA compreende o ciclo completo da formação de uma equipe, desde o registro das competências dos colaboradores até o acompanhamento dos resultados da equipe formada. Fazem parte do sistema:

- **Gerenciamento de colaboradores:** cadastro manual por formulário ou importação assistida a partir de currículos em PDF, com extração automática de habilidades e experiências.
- **Geração de equipes por IA:** composição automática a partir dos critérios informados pelo gestor (tamanho da equipe, habilidades exigidas, peso da experiência), com cálculo do Coeficiente de Aproveitamento para cada sugestão.
- **Ajuste manual com retroalimentação:** interface para substituir, incluir ou remover membros de uma sugestão; cada alteração é registrada e utilizada como *feedback* para o modelo.
- **Relatórios e indicadores:** visualização gráfica da qualidade das equipes e relatórios de desempenho individual e coletivo.
- **Sugestão de treinamentos:** recomendação de capacitações associadas às lacunas identificadas no perfil dos colaboradores.
- **Controle de acesso por perfil:** autenticação e autorização diferenciadas para os perfis Gestor/RH e Colaborador, com tratamento dos dados pessoais conforme a LGPD.

Estão explicitamente **fora do escopo** deste projeto:

- Processos de RH não relacionados à composição de equipes, como folha de pagamento, controle de ponto, recrutamento externo e avaliação de desempenho formal.

- Gestão de projetos e de tarefas: o sistema forma a equipe, mas não acompanha a execução do trabalho que ela realiza.
- Integração com sistemas de RH ou ERP de terceiros, prevista apenas por meio da exportação de relatórios em formatos abertos.
- Aplicativo móvel nativo; o acesso se dá por navegador, com interface responsiva.
- Qualquer decisão automática de contratação, promoção ou desligamento. O sistema é uma ferramenta de apoio à decisão, e a validação humana é sempre obrigatória.

1.3 Visão geral dos requisitos

Orientação: *Apresentem os requisitos funcionais e não funcionais mais importantes para compreender o projeto. Caso a equipe tenha uma especificação detalhada de requisitos produzida anteriormente, ela pode ser incluída em um apêndice ou referenciada neste documento.*

A especificação detalhada de requisitos, incluindo regras de negócio, casos de uso e histórias de usuário, foi elaborada na disciplina anterior e permanece válida como documento de referência. Esta seção reúne apenas os requisitos que influenciam diretamente as decisões de projeto discutidas neste documento.

A Tabela 1.2 apresenta os requisitos funcionais essenciais.

Tabela 1.2: Requisitos funcionais essenciais.

ID	Descrição
RF01	O sistema deve coletar e armazenar informações relevantes sobre os colaboradores, como habilidades, experiências, histórico de desempenho e lacunas de formação.
RF02	O sistema deve gerar automaticamente equipes balanceadas a partir dos atributos dos colaboradores e dos critérios definidos pelo gestor.
RF03	O sistema deve calcular e exibir o Coeficiente de Aproveitamento de cada equipe sugerida, permitindo a comparação entre composições alternativas.
RF04	O sistema deve permitir a troca manual de membros nas equipes sugeridas e registrar cada ajuste como <i>feedback</i> para o modelo de IA.
RF05	O sistema deve identificar as lacunas de cada colaborador e sugerir treinamentos correspondentes.
RF06	O sistema deve gerar relatórios de desempenho individual e coletivo e apresentar a qualidade das equipes de forma gráfica.
RF07	O sistema deve restringir o acesso às funcionalidades conforme o perfil do usuário (Gestor/RH ou Colaborador).

Os requisitos não funcionais mais determinantes para a arquitetura estão resumidos na Tabela 1.3, acompanhados da métrica adotada para verificá-los.

Vale destacar o RNF07, que não decorre de exigência do cliente, mas da natureza experimental do componente de IA: como o modelo será ajustado repetidamente ao longo da vida do sistema, tratá-lo como dependência substituível é uma decisão que atravessa toda a arquitetura descrita no Capítulo 2.

Tabela 1.3: Requisitos não funcionais determinantes para o projeto.

ID	Descrição	Métrica de verificação
RNF01	As senhas dos usuários devem ser armazenadas de forma criptografada.	Uso de algoritmo de <i>hash</i> forte (<i>bcrypt</i>); nenhuma senha em texto claro na base de dados.
RNF02	O tratamento de dados pessoais deve seguir a LGPD.	Ausência de vulnerabilidades de severidade alta ou crítica em teste de intrusão; acesso aos dados restrito por perfil.
RNF03	O sistema deve responder à solicitação de formação de equipe em tempo adequado.	Mais de 90% das equipes formadas dentro do tempo esperado, independentemente do tamanho do time.
RNF04	O sistema deve suportar aumento de carga de usuários simultâneos.	Tempo médio de resposta ≤ 2 s e taxa de erro $\leq 1\%$ em teste de carga com 50.000 conexões simultâneas.
RNF05	A interface deve ser utilizável por gestores sem treinamento especializado.	Pontuação superior a 70 na escala SUS e ajuste manual de equipe realizável em até três cliques.
RNF06	O sistema deve funcionar nos principais navegadores e resoluções de tela.	Renderização correta em Chrome, Firefox e Edge, e em resoluções a partir de 1024 px, sem instalação de <i>plugins</i> .
RNF07	O modelo de IA deve poder ser retreinado e substituído sem alteração da lógica de negócio.	Troca da implementação do serviço de IA sem modificar as classes de negócio, validada por testes com <i>mocks</i> .

1.4 Regras de negócio

Além dos requisitos, um conjunto de regras de negócio restringe o comportamento do sistema, definindo quando uma ação é permitida, o que deve ser registrado e como reagir a dados incompletos; levantadas na etapa anterior do projeto, permanecem válidas e são resumidas na Tabela 1.4.

RN02 e RN07 merecem destaque por gerarem diretamente casos de teste: a primeira define a condição de contorno entre colaborador elegível e inelegível, e a segunda exige que o sistema nunca sobrescreva uma composição anterior sem deixar rastro; ambas reaparecem como critério de aceite no Capítulo 5.

Tabela 1.4: Regras de negócio do RANK-IA.

ID	Descrição
RN01	Os colaboradores devem ter seus dados de competências, habilidades e experiências atualizados periodicamente pelo setor de RH, garantindo precisão nas recomendações da IA.
RN02	Somente colaboradores com dados completos podem ser considerados na formação automatizada de equipes.
RN03	Toda alteração manual realizada pelo RH ou gestor deve ser registrada, contendo motivo e mudanças feitas; esses dados alimentam o modelo de IA.
RN04	O acesso é restrito por perfil: RH e gestores têm acesso total ao sistema; colaboradores, apenas às próprias informações e aos treinamentos sugeridos.
RN05	Cada equipe formada deve gerar automaticamente seu Coeficiente de Aproveitamento, calculado com base em métricas definidas previamente.
RN06	O sistema deve gerar relatórios de desempenho das equipes e de evolução individual dos colaboradores e, a partir dos <i>gaps</i> evidenciados, sugerir treinamentos específicos.
RN07	Cada equipe gerada deve ser registrada como uma versão, permitindo rastrear modificações ao longo do tempo e garantindo auditoria do processo.

1.5 Objetivos de qualidade

Orientação: *Identifiquem os atributos de qualidade que realmente influenciam as decisões de projeto, por exemplo: manutenibilidade, desempenho, segurança, confiabilidade, usabilidade ou portabilidade. Podem ser utilizados modelos como ISO/IEC 25010 ou FURPS+, desde que apenas os atributos relevantes ao sistema sejam selecionados.*

Os objetivos de qualidade do RANK-IA foram definidos a partir do modelo de qualidade de produto da norma ISO/IEC 25010 [International Organization for Standardization, 2011] (Figura 1.1): das oito características previstas, foram selecionadas apenas as que efetivamente restringem as decisões de projeto, e as demais foram consideradas satisfeitas pelas práticas usuais de desenvolvimento web, sem discussão neste documento.

Functional Suitability	Reliability	Security	Maintainability
System provides functions that meet stated or implied needs.	System can maintain a specified level of performance when used under specified conditions.	Protection of system items from accidental or malicious access, use, modification, destruction, or disclosure.	System can be modified, corrected, adapted or improved due to changes in environment or requirements.
Performance Efficiency	Usability	Compatibility	Transferability
System provides appropriate performance, relative to the amount of resources used.	System can be understood, learned, used and is attractive to users.	Two or more systems can exchange information while sharing the same environment.	System can be transferred from one environment to another.

Figura 1.1: Características do modelo de qualidade de produto da ISO/IEC 25010.

A Tabela 1.5 sintetiza os objetivos priorizados e o que cada um implica concretamente no projeto do sistema.

Tabela 1.5: Objetivos de qualidade do RANK-IA.

Objetivo	Prioridade	Impacto esperado no projeto
Manutenibilidade	Alta	Modularização por coesão em torno do domínio de equipes, dependências expressas por interfaces (inversão de dependência) e testes automatizados. É o que torna viável substituir o modelo de IA sem reescrever a lógica de negócio.
Adequação funcional	Alta	A qualidade da sugestão é o valor central do produto: o Coeficiente de Aproveitamento precisa ser calculado de forma correta e explicável, e a sugestão deve ser aproveitável sem que o gestor a descarte por completo.
Segurança	Alta	Autenticação, autorização por perfil, criptografia de senhas e das comunicações, e segregação dos dados pessoais dos colaboradores em conformidade com a LGPD.
Confiabilidade	Alta	Persistência garantida das equipes salvas e dos ajustes manuais, com versionamento das composições para auditoria; para uma mesma entrada e um mesmo modelo, a sugestão deve ser reproduzível.
Usabilidade	Média	Interface consistente, ajuste manual em poucas etapas e apresentação gráfica dos indicadores, de modo que o gestor compreenda a composição sugerida sem conhecimento técnico sobre o modelo.
Eficiência de desempenho	Média	Separação do processamento da IA em serviço próprio, evitando que o custo da inferência degrade as demais operações da aplicação.
Portabilidade e compatibilidade	Baixa	Aplicação web responsiva, sem dependência de <i>plugins</i> , e leitura e exportação em formatos abertos (PDF, CSV).

Confiabilidade e Compatibilidade ainda carecem de meta numérica explícita neste documento: a primeira é medida, no plano de qualidade da equipe, por uma disponibilidade de serviço (*uptime*) de 99,5% durante o horário comercial e por um MTBF (tempo médio entre falhas) superior a 720 horas de operação contínua; a segunda, por uma taxa de sucesso de 99% na leitura de currículos em PDF gerados por diferentes versões de editor, já que a extração de currículos é o ponto do sistema mais exposto a variação no formato de entrada. Essas metas, somadas às já registradas nas Tabelas 1.3 e 1.5, são o critério de aceite que o Capítulo 5 usa para os testes de sistema e de compatibilidade.

Os dois primeiros objetivos tensionam entre si: a adequação funcional pressiona a equipe a experimentar com o modelo de IA, gerando código acoplado ao algoritmo do momento, enquanto a manutenibilidade exige o contrário. A escolha do projeto foi privilegiar a manutenibilidade na fronteira entre o domínio e a IA — isolando o modelo atrás de uma abstração — de modo que a experimentação fique contida em um único componente substituível, com consequências detalhadas no Capítulo 2 e no Capítulo 3.

Capítulo 2

Arquitetura do Sistema

Orientação: Apresentem a arquitetura proposta e justifiquem as principais decisões. O leitor deve conseguir compreender a estrutura geral do sistema antes de entrar nos detalhes de seus componentes.

2.1 Contexto e fronteiras do sistema

Orientação: Apresentem um diagrama de contexto mostrando o sistema, seus principais usuários e os sistemas externos com os quais ele se comunica. Expliquem as relações mostradas no diagrama.

2.2 Estilo ou padrão arquitetural

Orientação: Identifiquem o(s) estilo(s) ou padrão(ões) arquitetural(is) adotado(s), quando aplicável, e justifiquem a escolha considerando as características do sistema e seus objetivos de qualidade.

2.3 Contêineres e componentes

Orientação: Apresentem uma visão de alto nível da decomposição do sistema. Pode-se utilizar, quando adequado, os níveis de contexto, contêiner e componente do modelo C4. Não basta inserir os diagramas: descrevam a responsabilidade dos elementos relevantes e as principais dependências ou comunicações entre eles.

2.4 Decisões arquiteturais relevantes

Orientação: Registrem as decisões que tiveram impacto significativo na arquitetura. Para cada decisão, apresentem de forma breve a motivação, alternativas consideradas quando pertinente e principais consequências.

Tabela 2.1: Modelo para registrar decisões arquiteturais.

Decisão	Motivação	Consequências
Exemplo	Por que essa alternativa foi escolhida?	Benefícios, limitações ou compromissos envolvidos.

2.5 Restrições de arquitetura

Orientação: Registrem restrições que limitem as decisões de arquitetura ou implementação, como tecnologias obrigatórias, integrações com sistemas existentes, ambiente de execução, restrições organizacionais ou convenções. Incluam apenas restrições efetivamente relevantes ao projeto.

Capítulo 3

Projeto de Componentes

Orientação: Detalhem a organização interna das partes mais relevantes do sistema. O projeto de componentes pode ser representado graficamente, textualmente ou por tabelas, de acordo com a natureza da solução.

3.1 Visão geral dos componentes

Orientação: Listem os principais componentes e suas responsabilidades. Para sistemas maiores, esta visão geral pode cobrir todos os componentes, enquanto o detalhamento das seções seguintes pode se concentrar apenas nos mais relevantes ou complexos.

Tabela 3.1: Principais componentes do sistema.

Componente	Responsabilidade principal
Componente A	Descrever sua responsabilidade principal.
Componente B	Descrever sua responsabilidade principal.

3.2 Detalhamento dos componentes principais

Orientação: Seleccionem os componentes mais relevantes ou complexos e apresentem sua estrutura, responsabilidades, interfaces e dependências. Se o projeto for orientado a objetos, utilizem principalmente diagramas de classes UML e, quando necessário, complementem com diagramas de sequência, atividade ou outros artefatos.

3.3 Princípios de projeto

Orientação: Expliquem como os princípios de projeto discutidos na disciplina foram aplicados nos principais componentes ou classes do sistema. Por exemplo, podem ser discutidos aspectos relacionados à separação de responsabilidades, coesão, acoplamento, dependência de abstrações e princípios SOLID. Não apenas enumerem os princípios: indiquem onde foram aplicados e qual problema de projeto ajudaram a resolver. Para evidenciar sua aplicação, utilizem, quando adequado, diagramas de classes ou trechos de código-fonte.

3.4 Padrões de projeto

Orientação: Para cada padrão de projeto efetivamente utilizado, indiquem o problema que motivou sua aplicação, as classes ou componentes participantes, como o padrão foi aplicado e qual benefício se espera obter. Não utilizem padrões apenas para preencher esta seção.

Capítulo 4

Projeto de Interface do Usuário

Orientação: *Apresentem as interfaces e os principais fluxos de interação do usuário com o sistema. O principal artefato pode ser constituído por protótipos de baixa ou alta fidelidade. Diagramas de atividade, fluxos de navegação ou outros artefatos podem complementar a explicação.*

Não é necessário apresentar todas as telas. Priorizem as interfaces e os fluxos mais importantes para compreender o funcionamento do sistema.

Orientação: *Para cada fluxo principal, expliquem brevemente a tarefa do usuário e as decisões de interface relevantes. Se a equipe utilizar uma ferramenta externa, como Figma, apresentem exemplos representativos neste documento e forneçam o link para o material completo.*

Capítulo 5

Especificação e Estratégia de Testes

***Orientação:** Mostrem como a equipe pretende verificar o comportamento do software e reduzir o risco de defeitos. O capítulo deve apresentar uma estratégia coerente, e não apenas uma lista de ferramentas.*

A estratégia de testes do RANK-IA acompanha a divisão em quatro processos e um banco de dados descrita no Capítulo 6: o *frontend*, a API e os dois serviços em Python (IA Geradora de Equipes e IA Analista de Currículos), além do PostgreSQL. Cada nível de teste cobre um tipo de defeito que os outros não alcançam, e nenhum substitui os demais.

A estratégia também separa dois papéis complementares: verificação, que responde à pergunta “estamos construindo o software corretamente?”, e validação, que responde a “estamos construindo o software certo?”. Os níveis de unidade, integração e contrato, descritos nas próximas seções, respondem à primeira pergunta; a validação e a aceitação, tratadas na Seção 5.4, respondem à segunda. Como a equipe é pequena e os mesmos quatro integrantes desenvolvem e testam, o RANK-IA não conta com um grupo de teste independente separado; o papel que caberia a esse grupo — avaliar o código sem o viés de quem o escreveu — é cumprido pela exigência, já registrada no Capítulo 6, de que todo *pull request* seja aprovado por alguém que não seja o autor.

5.1 Planejamento e níveis de teste

***Orientação:** Indiquem os níveis de teste aplicáveis ao projeto, por exemplo, unidade, integração e sistema, e expliquem o objetivo de cada um deles no contexto do software.*

Testes de unidade verificam uma função ou classe isolada, sem depender de rede, banco ou dos demais serviços; são o nível mais barato de manter e o primeiro a apontar um defeito, o que os torna a base da pirâmide. Testes de integração verificam se a API de fato consegue conversar com os dois serviços de IA e com o banco, sem que o comportamento correto de um dependa de uma suposição errada sobre o outro.

Entre esses dois níveis, o projeto adota também um nível de **teste de contrato**: como o RNF07 promete que o modelo de IA pode ser substituído sem alterar a lógica de negócio, o que precisa ser validado não é uma implementação específica, mas o contrato da interface `IIAGeradoraService` em si, cuja mesma bateria de testes deve passar independentemente de qual serviço a implementa no momento. Sem esse nível, a promessa do RNF07 fica só na intenção, porque nada garante que uma troca de implementação não quebre silenciosamente uma suposição que só existia na cabeça de quem escreveu a versão anterior.

Depois que o software está integrado, dois níveis de ordem superior fecham a pirâmide. A validação, tratada na Seção 5.4, confirma que o sistema atende aos requisitos do ponto de vista do usuário, exercitando os fluxos principais e alternativos já descritos nos casos de uso

UC-01 (Gerenciar Colaboradores), UC-02 (Gerar Equipes com IA) e UC-03 (Ajustar Equipe Manualmente), documentados na etapa anterior do projeto. O teste de sistema, tratado na Seção 5.5, verifica o RANK-IA já combinado aos demais elementos do ambiente (carga, falha de um serviço, segurança, configuração), e é o que dá sustância às metas quantitativas de Confiabilidade e Compatibilidade fechadas na Seção 1.5, que de outra forma ficariam sem verificação correspondente.

5.2 Testes de unidade

***Orientação:** Seleccionem unidades relevantes e definam cenários e casos de teste. Quando existirem dependências externas ou difíceis de controlar, expliquem se serão utilizados stubs, mocks ou outras formas de substituição. Apresentem exemplos representativos contendo entradas, condições e resultados esperados.*

As unidades priorizadas para teste são as que concentram regra de negócio, não as que só encaminham dados: o cálculo do Coeficiente de Aproveitamento (RN05), a validação de elegibilidade de um colaborador para entrar na geração automática (RN02), o controle de acesso por perfil (RN04) e o registro de uma nova versão de equipe (RN07). Dependências externas ao módulo testado — o módulo de IA, o serviço de e-mail, o banco — são substituídas por *mocks* que respeitam as interfaces já definidas (`IIAGeradoraService`, `INotificadorService`), possível porque essas dependências foram isoladas atrás de abstrações desde o projeto de componentes (o pagamento concreto da manutenibilidade do Capítulo 3), como mostra a Tabela 5.1.

Tabela 5.1: Exemplos de casos de teste de unidade.

Unidade	Entrada / condição	Resultado esperado
Cálculo do CA	Equipe com todos os membros possuindo as habilidades requeridas	CA no valor máximo definido pela escala
Cálculo do CA	Um membro sem nenhuma das habilidades requeridas	CA reduzido de forma proporcional, nunca negativo
Elegibilidade (RN02)	Colaborador com cadastro incompleto (habilidade sem nível)	Colaborador excluído do conjunto elegível, sem erro não tratado
Controle de acesso (RN04)	Colaborador autenticado tenta acessar rota de gestão de equipes	Acesso negado, sem exposição de dados de outros colaboradores
Versionamento (RN07)	Equipe existente recebe um ajuste manual	Nova versão criada; versão anterior permanece recuperável

Uma técnica complementa os casos fixos da Tabela 5.1 no módulo de cálculo do CA, por ser o de maior risco de negócio (é a Adequação Funcional priorizada como Alta na Seção 1.5): o **teste baseado em propriedades**. Em vez de fixar entradas, o teste declara invariantes que devem valer para qualquer entrada válida (por exemplo, “substituir um membro por outro com competências estritamente superiores nunca reduz o CA da equipe”), e uma ferramenta gera automaticamente combinações de colaboradores tentando violar essa propriedade.

5.2.1 Test-Driven Development

No módulo de cálculo do CA e nas regras de negócio da Tabela 1.4, a equipe adota TDD: o teste é escrito antes do código de produção, falha por não existir implementação, e só então é

escrito o código mínimo necessário para passar, seguido de refatoração. A adoção não é ampla: código de integração com os serviços de IA e telas de interface continuam sendo testados depois de escritos, porque o custo de simular a resposta da IA antes de decidir o formato da integração seria maior que o benefício, e TDD fica reservado à lógica de domínio — justamente onde escrever o teste primeiro força o desenho a ser testável e desacoplado, pagando na prática o objetivo de manutenibilidade do Capítulo 2.

5.3 Testes de integração

Orientação: *Expliquem como os componentes serão integrados e testados. Quando fizer sentido, indiquem a estratégia adotada (por exemplo, top-down, bottom-up, depth-first ou breadth-first) e justifiquem-na. Descrevam também como testes de regressão serão utilizados após alterações.*

A integração segue estratégia *top-down* a partir da API, que é o componente que orquestra os dois serviços de IA e o banco: primeiro se valida que a API conversa corretamente com cada serviço isoladamente (contrato), depois que a cadeia completa — requisição do gestor, processamento da IA, persistência — produz o resultado esperado. A opção por *top-down* em vez de *bottom-up* é intencional: é a API que concentra a regra de negócio, e um defeito de integração que afete o gestor quase sempre aparece primeiro ali, não nos serviços Python isolados.

Os cenários de integração priorizados vêm diretamente dos fluxos alternativos dos casos de uso levantados na etapa anterior:

- **Falha na leitura do PDF** (UC-01): currículo corrompido, digitalizado como imagem sem camada de texto, em formato inesperado ou vazio. Em todos os casos, a API deve responder com uma falha tratada e sugerir o cadastro manual, nunca propagar uma exceção não tratada do serviço de IA para o *frontend*.
- **Colaboradores insuficientes** (UC-02): base de dados sem colaboradores com as habilidades exigidas. A API deve notificar a impossibilidade de forma explícita, em vez de devolver uma equipe incompleta silenciosamente.
- **Cancelar ajuste** (UC-03): o gestor reverte um ajuste manual antes de confirmar. A composição original da IA deve ser restaurada sem que o cancelamento seja registrado como *feedback* para o modelo.

A regressão é responsabilidade da esteira de integração contínua, não de uma execução manual: a cada *pull request* para a *main*, o trabalho “Testes de integração” descrito na Tabela 6.3 sobe um PostgreSQL efêmero e executa essa suíte por completo, de modo que uma alteração no serviço de IA que quebre um contrato já validado é pega antes da revisão humana, não depois dela.

5.4 Teste de validação e aceitação

Orientação: *Descrevam como a equipe verifica se o software atende aos requisitos do ponto de vista do usuário, e como está prevista a aceitação por parte de quem vai efetivamente usar o sistema.*

A validação confirma que o RANK-IA integrado atende aos requisitos funcionais da Tabela 1.2 e às regras de negócio da Tabela 1.4 do ponto de vista de quem vai usar o sistema, não de quem o escreveu. O critério de validação, portanto, não é “o código faz o que o desenvolvedor pretendia”, mas “o sistema faz o que o RF ou a RN descrevem”; quando os dois divergem, quem está certo é o requisito.

A aceitação segue o mesmo desenho de dois ambientes já descrito no Capítulo 6: o ambiente de homologação cumpre o papel do que a literatura de teste chama de teste alfa, pois a cada integração na *main* a equipe (e, quando disponível, um representante do RH que participou do levantamento de requisitos) usa o incremento nesse ambiente antes da liberação, sob observação direta de quem desenvolveu, registrando tanto falhas quanto dificuldades de uso que não chegam a configurar um defeito. Um teste beta formal, conduzido sem a presença da equipe em instalações do cliente, não se aplica ainda a este projeto: o RANK-IA não tem organização cliente em produção, e a validação com usuário externo fica registrada aqui como prática a adotar assim que houver uma organização piloto, não como algo já executado.

5.5 Teste de sistema

Orientação: *Descrevam como o software será testado já combinado aos demais elementos do sistema (hardware, rede, pessoas, dados), cobrindo aspectos como recuperação de falhas, segurança, desempenho sob carga e configuração.*

Testado como parte de um sistema maior — que inclui o banco de dados, a rede e as pessoas que o operam —, o RANK-IA passa por cinco verificações de ordem superior, cada uma expondo um tipo de falha que os níveis anteriores não alcançam.

Teste de recuperação. Um roteiro derruba de propósito o contêiner da IA Geradora de Equipes em ambiente de homologação, no meio de uma requisição, e verifica se a API degrada para o fluxo de cadastro/composição manual (UC-01, UC-03) em vez de retornar erro ao gestor, e se a recuperação automática do contêiner devolve o serviço sem intervenção humana. É a única forma de testar na prática, e não só na interface, a substituíbilidade prometida pelo RNF07.

Teste de segurança. Segue a divisão já registrada na Tabela 1.3: uma varredura automatizada (OWASP ZAP, modo linha de base) roda contra as rotas de *upload* de currículo a cada *release*, e o teste de penetração completo permanece com a equipe de Pentest, fora da automação, exatamente como o RNF02 já prevê.

Teste por esforço. Um cenário de k6 simula as 50 mil conexões simultâneas da meta de Escalabilidade (Tabela 1.3), medindo se o tempo de resposta médio e a taxa de erro permanecem dentro do limite definido. Um segundo cenário, de sensibilidade, gera currículos com volumes extremos de habilidades listadas e solicita equipes com o número máximo de membros permitido, para verificar se uma entrada dentro dos limites válidos, mas próxima da fronteira, não degrada desproporcionalmente o cálculo do CA.

Teste de desempenho. Medido em conjunto com o teste por esforço, mas também sob carga normal: o tempo entre o gestor solicitar a geração de uma equipe e a resposta com as sugestões é o que a meta de Desempenho da Tabela 1.3 fixa em mais de 90% das equipes formadas dentro do tempo esperado.

Teste de disponibilização. Executa a suíte de sistema em cada combinação de navegador e resolução previstas na RNF06 (Chrome, Firefox, Edge; a partir de 1024 px), e valida que o *script* de carga inicial de dados sintéticos e as migrações rodam sem intervenção manual em uma instalação limpa do ambiente Docker descrito no Capítulo 6.

Como as etapas de recuperação, esforço e disponibilização são caras para rodar a cada *commit*, ficam fora da esteira padrão de *pull request* e são executadas em horário agendado ou antes de uma *release*, seguindo a mesma lógica de custo que já separa a esteira de integração contínua da liberação em produção no Capítulo 6.

5.6 Critérios de conclusão e cobertura

Orientação: *Definam critérios que permitam decidir quando a atividade de teste atingiu um nível satisfatório. A cobertura de código pode ser um dos indicadores, mas não deve ser tratada isoladamente como garantia de qualidade. Se uma meta quantitativa for adotada, indiquem o tipo de cobertura e justifiquem a meta.*

A atividade de teste é considerada satisfatória quando três condições se verificam ao mesmo tempo, e não quando qualquer uma delas isoladamente é atingida. Primeiro, cobertura: 80% de cobertura de linha no módulo de domínio (cálculo do CA, regras de elegibilidade e versionamento), sem meta equivalente para código de interface, onde cobertura alta tende a testar o *framework* em vez da lógica da equipe. Segundo, rastreabilidade: todo requisito funcional da Tabela 1.2 tem ao menos um teste de validação associado (Seção 5.4), e toda regra de negócio da Tabela 1.4 tem ao menos um teste de unidade ou integração associado, estendendo para o nível de teste a mesma cadeia de rastreabilidade da Seção 6.3, que hoje termina no caso de teste sem detalhar o que isso significa. Terceiro, metas quantitativas: cada meta numérica de RNF ou de objetivo de qualidade (Tabelas 1.3 e 1.5) tem um teste de sistema correspondente, descrito na Seção 5.5, que a exercita de forma direta.

Cobertura de código entra nesses critérios como piso, não como alvo: um módulo pode chegar a 100% de cobertura de linha exercitando apenas o caminho de sucesso, sem que isso diga nada sobre o comportamento diante de um currículo corrompido ou de uma base sem colaboradores elegíveis. Por isso, a meta de 80% é sempre lida em conjunto com os cenários de exceção da Seção anterior, nunca isoladamente.

5.7 Depuração

Orientação: *Descrevam, ainda que brevemente, como a equipe pretende investigar e corrigir os defeitos revelados pelos testes, e não apenas como os testes serão executados.*

A depuração começa onde o teste termina: um caso de teste que falha aponta um sintoma, não necessariamente a causa, e conectar os dois é o trabalho de depurar. A equipe evita a depuração por força bruta — inundar o código de instruções de saída na esperança de tropeçar na causa — em favor da eliminação de causa: formular uma hipótese específica sobre a origem do defeito a partir do sintoma e do caso de teste que o revelou, e usar o depurador para confirmá-la ou descartá-la antes de alterar qualquer linha. Como a equipe não conta com um grupo de teste independente, a revisão de *pull request* cumpre também esse papel na depuração: o revisor, por não ter escrito a correção, tem mais chance de perceber quando ela é sintomática demais e mascara a causa real.

Antes de integrar uma correção, a equipe responde às três perguntas que Van Vleck propõe para qualquer defeito: a causa se repete em outro ponto do sistema? Que novo defeito essa correção pode introduzir? O que teria evitado esse defeito desde o início? É a terceira pergunta que efetivamente muda o processo: quando a resposta aponta para a ausência de um teste, esse teste é escrito antes do *merge*, não depois, para que o mesmo defeito não precise ser depurado duas vezes.

5.8 Automação e ferramentas

Orientação: *Descrevam as ferramentas utilizadas ou planejadas para executar testes, gerar relatórios e, quando aplicável, automatizar sua execução.*

As ferramentas planejadas por nível de teste são:

- **Unidade (*frontend*/API):** Vitest e React Testing Library no *frontend*; Vitest e *mocks* manuais das interfaces de domínio na API.
- **Unidade (serviços Python):** pytest; Hypothesis para os testes baseados em propriedades do cálculo do CA.
- **Integração:** Supertest contra a API, com PostgreSQL efêmero subido pela esteira (Tabela 6.3); *stubs* dos serviços de IA para os cenários de falha que seriam caros de reproduzir de verdade (ex. currículo corrompido).
- **Contrato:** verificação de esquema (JSON Schema) da entrada e saída de `IIAGeradoraService` e `INotificadorService`, executada contra qualquer implementação candidata antes de ela substituir a atual.
- **Sistema / aceitação:** Playwright, cobrindo os fluxos principais e alternativos de UC-01, UC-02 e UC-03 em navegador real.

Além dos níveis clássicos, um *script* agendado cobre o que é específico do componente de IA e não se encaixa nas categorias tradicionais de teste de sistema da Seção 5.5: depois de cada re-treinamento semanal da IA (História RIA-03), ele compara a distribuição de CA gerada pelo modelo novo contra a do anterior para o mesmo conjunto de colaboradores de teste e alerta se a mudança ultrapassar um limiar, pegando uma regressão de qualidade antes que ela afete equipes reais. O mesmo *script* gera equipes variando apenas um atributo pessoal do colaborador (mantendo habilidades e desempenho idênticos) e verifica se o CA muda: como a Seção de Escopo do Capítulo 1 já deixa qualquer decisão automática de contratação, promoção ou desligamento fora do sistema, este teste é a contrapartida técnica dessa decisão, não decidindo se um viés existe, mas garantindo que, se existir, seja visível antes de chegar ao gestor.

Por fim, os relatórios de teste seguem o mesmo princípio de rastreabilidade do Capítulo 6: o relatório de cobertura é publicado como comentário no *pull request* (Tabela 6.3), e os resultados dos testes de sistema (Seção 5.5) e dos *scripts* desta seção são anexados à *release* correspondente, junto às notas de versão. Assim, para qualquer versão entregue, é possível responder não só o que mudou, mas também o que foi verificado antes de mudar.

Capítulo 6

Gestão de Configuração e Manutenção

***Orientação:** Este capítulo deve mostrar como os artefatos do software são versionados, modificados e mantidos de forma controlada. Relacionem as práticas descritas ao projeto da equipe, evitando apenas reproduzir definições teóricas.*

O RANK-IA é desenvolvido por uma equipe pequena, distribuída entre partes distintas do sistema, e entregue em ciclos de duas semanas a uma organização que depende dele para formar suas equipes de trabalho. Essa combinação exige três garantias da gestão de configuração: que qualquer versão já entregue possa ser reconstruída exatamente como foi publicada, para investigar um problema relatado por um gestor sem depender do que alguém lembra ter feito; que toda alteração esteja ligada à necessidade que a motivou; e que o conhecimento sobre como o sistema é construído esteja registrado, não apenas na cabeça de quem escreveu cada parte, já que a substituição do modelo de IA prevista pelo RNF07 e o objetivo de manutenibilidade da Seção 1.5 pressupõem que outra pessoa consiga mexer nesse código depois.

Dois ferramentas sustentam esse controle: o **GitHub** hospeda o repositório, as revisões de código, a documentação de apoio e as automações, e o **Taiga.io**, já adotado pela equipe como quadro Scrumban, organiza o trabalho e hospeda a wiki do projeto. A ligação entre as duas é o que fecha a cadeia de rastreabilidade descrita na Seção 6.3.

6.1 Repositório e itens de configuração

***Orientação:** Informem o repositório utilizado e identifiquem os principais itens de configuração que precisam ser controlados, por exemplo: código-fonte, arquivos de configuração, scripts, documentação, modelos de dados e testes.*

O controle de versões é feito com **Git**, em repositório único (*monorepo*) hospedado no GitHub em <https://github.com/hiroshimurosaki/rank-ia>. A alternativa seria um repositório por contêiner, mas isso não se justifica aqui: com uma equipe deste tamanho, é comum que uma única funcionalidade atravesse o *frontend*, a API e o serviço de IA, e manter essa alteração em um só *commit* evita que as partes do sistema entrem em versões incompatíveis entre si — o custo típico do *monorepo*, acoplar ciclos de entrega que deveriam ser independentes, só aparece em produtos bem maiores que este.

O GitHub foi escolhido pela familiaridade da equipe e por três facilidades concretas: revisão de código por *pull request* com o histórico preservado, execução das automações no próprio GitHub Actions sem infraestrutura extra, e integração nativa com o Taiga.io, que a equipe já usava desde a disciplina anterior.

A Tabela 6.1 identifica os itens de configuração sob controle.

Tabela 6.1: Itens de configuração do RANK-IA.

Item	Conteúdo e observações
Código-fonte da aplicação	<i>Frontend</i> React/TypeScript, API Node.js/TypeScript e os dois serviços em Python (Engine de IA e leitor de currículos).
Comentários e documentação interna	Comentários no código e anotações de interface fazem parte do código-fonte e são revisados junto com ele.
Testes automatizados	Testes de unidade e de integração, versionados junto ao código que exercitam. Um <i>commit</i> que altera comportamento e não toca em teste algum é sinalizado na revisão.
Esquema do banco de dados	Arquivos de migração versionados e imutáveis após a integração: corrigir uma migração já aplicada exige uma nova migração, nunca a edição da anterior.
Descritores de dependências	<code>package.json</code> e <code>package-lock.json</code> nos projetos Node, <code>requirements.txt</code> com versões fixadas nos serviços Python.
Configuração de ambiente	<code>docker-compose.yml</code> , <code>Dockerfile</code> de cada serviço e <code>.env.example</code> com a lista de variáveis exigidas, sem valores.
Automações	Fluxos de trabalho do GitHub Actions em <code>.github/workflows/</code> e os <i>scripts</i> auxiliares invocados por eles.
Documentação de projeto	Este documento (fonte L ^A T _E X, <code>refs.bib</code> e imagens), <code>README.md</code> e os diagramas C4 e UML em formato editável, e não apenas exportados como imagem.
Wiki do projeto	Guia de uso do sistema e guia de contribuição, mantidos na wiki nativa do Taiga.io, com histórico de páginas.
Parametrização do modelo de IA	Critérios de formação, hiperparâmetros e identificador da versão do modelo em uso, em arquivo de configuração versionado.

6.1.1 Documentação em três camadas

A equipe trata documentação como um item de configuração em três níveis, cada um respondendo a uma pergunta diferente e mantido em um lugar diferente.

Este documento responde **por que** o sistema é como é: registra as decisões de arquitetura e de projeto e as razões por trás delas, e muda pouco, apenas quando uma decisão relevante é tomada ou revista.

Os **comentários no código** respondem por que um trecho específico faz o que faz, seguindo uma convenção deliberadamente restritiva: comentário que apenas repete em português o que a linha ao lado já diz é ruído, envelhece mal e acaba mentindo, então só se comenta o que o código não consegue dizer sozinho, como a razão de uma regra de negócio pouco óbvia, o motivo de uma solução de contorno ou a fórmula por trás do Coeficiente de Aproveitamento. Além disso, toda interface pública recebe anotação estruturada (TSDoc no código TypeScript, *docstring* nos serviços Python) descrevendo o contrato — o que cada parâmetro significa, o que é devolvido e em que condições o método falha —, o que torna uma abstração como `IIAGeradoraService` utilizável por quem não escreveu a implementação por trás dela.

A **wiki do projeto**, hospedada no próprio Taiga.io, responde **como** fazer as coisas, e é a camada que mais muda, com duas partes: a primeira é o guia de uso, escrito para o gestor de RH — como cadastrar colaboradores, importar currículos, solicitar a formação de uma equipe, interpretar o Coeficiente de Aproveitamento e ajustar uma composição sugerida —, e

a segunda é o guia de contribuição, escrito para quem vai programar, cobrindo as convenções já estabelecidas no projeto:

- como preparar o ambiente local e executar o sistema;
- a organização dos módulos e a regra de coesão que a orienta, de modo que código novo seja colocado onde os demais desenvolvedores esperam encontrá-lo;
- as convenções de nomenclatura e de estilo, e como executar as ferramentas de análise estática antes de abrir um *pull request*;
- quando criar uma abstração e como declarar dependências por interface, seguindo o que foi decidido no Capítulo 3;
- o padrão de mensagem de *commit* e o fluxo de *branches*;
- receitas para as tarefas recorrentes, como criar uma migração de banco, adicionar um endpoint na API ou escrever um teste com *mock* de uma interface.

A razão de existir dessa segunda parte é prática: boa parte do padrão do projeto não está escrita em lugar nenhum, mora no código já existente, e quem chega depois só o descobre por imitação ou por comentário em revisão de código. Escrever o padrão reduz o tempo que um novo integrante leva para produzir código consistente com o resto e diminui a discussão repetida nas revisões, que passam a se concentrar na lógica em vez da forma.

6.1.2 O que fica fora do repositório

Três categorias de artefato são deliberadamente mantidas fora do controle de versão. Os **segredos** (senhas de banco, chaves de API, o segredo de assinatura dos *tokens* JWT) são substituídos pelo `.env.example` e fornecidos em tempo de execução por variáveis de ambiente, e os **artefatos gerados** (`node_modules`, *bundles* do *frontend*, imagens de contêiner) são reconstrutíveis a partir do que está versionado, só ocupando espaço e poluindo o histórico.

A terceira categoria, a mais importante neste sistema, são os **dados pessoais de colaboradores**: currículos, competências e histórico de desempenho, que pelo RNF02 não podem sair do ambiente do cliente nem circular pelo repositório. Os ambientes de desenvolvimento e de homologação são populados por um *script* de carga com dados sintéticos, esse sim versionado, e nenhuma cópia da base de produção é usada para depuração.

Os pesos do modelo de IA treinado também não vão para o Git, por serem arquivos binários grandes e ilegíveis em *diff*: o que fica versionado é o *script* de treinamento, a configuração usada e o identificador da versão do modelo, e o arquivo de pesos correspondente é publicado como anexo de uma *release*. Assim continua sendo possível dizer, para qualquer versão do software em uso, qual modelo ela estava executando.

6.1.3 Organização de *branches* e versões

A equipe trabalha com *branches* curtos sobre a *branch* principal. A `main` contém apenas código integrado e aprovado pela verificação automatizada, e está protegida contra *push* direto: toda alteração entra por *pull request*. Cada tarefa do quadro Scrumban dá origem a um *branch* nomeado como `tipo/TG-<ref>-descricao-curta`, em que `tipo` é `feature`, `fix` ou `docs` e `<ref>` é o identificador da história ou tarefa no Taiga.

Fluxos mais elaborados, com um *branch* de desenvolvimento permanente separado do de produção, foram considerados, mas fazem sentido apenas quando várias versões do produto precisam ser mantidas simultaneamente, o que não é o caso do RANK-IA: existe uma versão em produção por vez, e correções urgentes seguem o mesmo caminho das demais alterações, apenas com prioridade maior na fila de revisão. Adotar o fluxo mais complexo acrescentaria uma etapa de sincronização sem benefício correspondente.

Ao final de cada *sprint*, a `main` recebe uma *tag* anotada seguindo versionamento semântico

(v0.1.0, v0.2.0 e assim por diante), e é essa *tag* que identifica o que foi entregue ao cliente. Recuperar o estado exato de uma versão publicada é, portanto, uma operação de um comando, o que importa tanto para reproduzir um defeito relatado quanto para retornar rapidamente à versão anterior se uma entrega apresentar problema.

6.2 Gestão de dependências e alterações

Orientação: *Expliquem como dependências externas são registradas e versionadas e como alterações relevantes são controladas. Quando uma alteração afetar vários artefatos, indiquem como a equipe mantém a coerência entre código, testes, documentação e demais elementos relacionados.*

As dependências externas são fixadas por arquivo de trava: nos projetos Node.js, o `package-lock.json` é versionado e a instalação nas automações usa `npm ci`, que instala exatamente as versões travadas e falha se `package.json` e `package-lock.json` estiverem divergentes; nos serviços Python, o `requirements.txt` registra versões exatas, não faixas. O resultado é que a construção do software é determinística: dois integrantes que constroem o mesmo *commit* em máquinas diferentes obtêm o mesmo conjunto de bibliotecas, e o que roda em produção é o que foi testado.

A atualização dessas dependências fica a cargo do Dependabot, configurado para abrir um *pull request* semanal por dependência desatualizada, passando pela mesma verificação automatizada que uma alteração escrita pela equipe. Uma atualização que quebre os testes aparece isolada, em um *pull request* próprio, em vez de ser descoberta no meio do desenvolvimento de outra funcionalidade.

O controle de alterações se apoia inteiramente no *pull request*: nenhum código chega à *main* sem passar por um, e cada um exige a aprovação de ao menos um integrante que não seja o autor. O modelo de *pull request* do repositório (`.github/pull_request_template.md`) traz uma lista de verificação curta, cujos itens são o que a equipe usa como *Definition of Done*:

- a alteração referencia a história ou tarefa correspondente no Taiga;
- testes automatizados cobrem o comportamento novo ou alterado;
- se um contrato entre componentes mudou, os comentários de interface e a documentação correspondente foram atualizados no mesmo *pull request*;
- se o esquema do banco mudou, existe uma migração acompanhando a alteração;
- se a alteração muda ou cria uma convenção do projeto, a wiki foi atualizada;
- a verificação automatizada está aprovada.

Os três itens do meio são a resposta da equipe ao problema da coerência entre artefatos. A alteração mais representativa nesse sentido é justamente a prevista pelo RNF07 — substituir o algoritmo de formação de equipes —, que atinge ao mesmo tempo a implementação do serviço de IA, o contrato da interface `IIAGeradoraService` e sua documentação, os *mocks* usados pelos testes de unidade do `GerenciadorDeEquipes`, a configuração de hiperparâmetros e a descrição do componente neste documento. A regra é que esses artefatos viajam juntos: um *pull request* que altere a interface sem atualizar os testes e a documentação não é aprovado — uma regra barata de seguir, e o que impede que a documentação se descole do código ao longo do tempo, que é a forma como documentação costuma morrer.

6.3 Rastreabilidade

Orientação: *Quando aplicável, mostrem como uma alteração pode ser rastreada entre os artefatos relacionados, por exemplo: necessidade/requisito, tarefa ou issue, componente afetado, commit e caso de teste. Não é necessário criar uma matriz extensa; um ou dois exemplos relevantes podem ser suficientes para demonstrar a prática.*

O planejamento do trabalho acontece no Taiga.io, onde ficam o *backlog*, as histórias de usuário, as tarefas e o quadro Kanban com as colunas *Novo*, *Em Andamento*, *Pronto para teste*, *Fechado* e *Precisa de informação*, enquanto o código, as revisões e as automações ficam no GitHub. Manter as duas ferramentas desconectadas criaria duas fontes de verdade que divergem em poucas semanas, o que a equipe evita com a integração de controle de versão do Taiga, configurada por *webhook* no repositório.

A ligação se dá pela mensagem de *commit*: toda mensagem cita a referência do item no Taiga no formato **TG-`<ref>`**, e o Taiga passa a exibir aquele *commit* no histórico da história ou tarefa correspondente. A mesma referência aparece no nome do *branch* e no título do *pull request*, de modo que a simples leitura do histórico do Git já revela a que necessidade cada alteração atende, enquanto as *issues* do GitHub são usadas para o que nasce fora do planejamento da *sprint* — tipicamente defeitos encontrados em revisão, em teste ou relatados por um usuário — e são espelhadas como *issues* no Taiga para que o quadro continue refletindo todo o trabalho em andamento, não apenas o que foi planejado.

Uma limitação já observada pela equipe merece registro: o Taiga gera os identificadores de histórias e tarefas de forma sequencial e automática, sem permitir que a equipe os defina, e por isso a numeração da plataforma não coincide com a usada nas primeiras atividades de modelagem. A referência adotada como oficial para efeito de rastreabilidade é a do Taiga, e é ela que aparece nas mensagens de *commit*.

A cadeia completa, portanto, é:

requisito → história de usuário → tarefa → *branch* → *commits* → *pull request* → componente
→ caso de teste

A Tabela 6.2 percorre essa cadeia em dois exemplos. Não se pretende construir uma matriz exaustiva; a intenção é mostrar que o caminho existe e é percorrível nos dois sentidos, tanto do requisito até o teste que o verifica quanto de um *commit* qualquer de volta até a necessidade que o originou.

Tabela 6.2: Exemplos de rastreabilidade entre artefatos.

Artefato	Exemplo 1	Exemplo 2
Requisito	RF04: troca manual de membros com registro de <i>feedback</i>	RNF01: senhas armazenadas de forma criptografada
História / <i>issue</i>	História <i>Troca manual de membros (Sprint 1)</i>	<i>Issue</i> aberta na revisão de segurança, espelhada no Taiga
Tarefa	Criação da função de troca manual; criação da interface da troca	Substituição do armazenamento da senha por <i>hash bcrypt</i>
<i>Branch</i>	feature/TG-14-troca-manual	fix/TG-57-hash-senha
Mensagem de <i>commit</i>	TG-14 registra ajuste manual como feedback do modelo	TG-57 aplica <i>bcrypt</i> no cadastro e na autenticação
Componente afetado	GerenciadorDeEquipes, HistoricoAjustes e a tela de ajuste	Serviço de autenticação e migração da tabela de usuários
Caso de teste	Ajuste manual altera a composição e persiste o registro de <i>feedback</i>	Senha não é recuperável a partir do valor armazenado; autenticação valida o <i>hash</i>

6.4 *Build*, integração e entrega

Orientação: *Descrevam como o software é construído e executado. Se houver automação de build, testes, integração contínua ou entrega/deploy, apresentem o fluxo adotado e as ferramentas envolvidas. Caso essas práticas não sejam utilizadas, expliquem brevemente como o processo é realizado manualmente.*

6.4.1 Construção local

O sistema é composto por quatro processos e um banco de dados, cada um com suas próprias exigências de versão de tempo de execução, de bibliotecas nativas e de configuração, e montar esse ambiente manualmente é demorado — mas o problema maior não é o tempo, é que cada integrante acabaria com uma combinação ligeiramente diferente de versões de Node.js, de Python e de PostgreSQL, além das particularidades do próprio sistema operacional. Basta uma dessas diferenças para que um defeito apareça na máquina de um desenvolvedor e não na do outro, e o esforço passa a ser gasto investigando o ambiente em vez do software.

É exatamente esse risco que o uso do Docker elimina. Ao descrever cada serviço em um `Dockerfile` e a composição entre eles no `docker-compose.yml`, o ambiente de execução deixa de ser uma característica da máquina de quem programa e passa a ser um item de configuração versionado como qualquer outro: todos os integrantes executam as mesmas versões, com as mesmas dependências de sistema, independentemente de estarem em Linux, Windows ou macOS, e um único comando levanta o *frontend*, a API, os dois serviços Python e a instância do PostgreSQL, com um *script* aplicando as migrações e carregando a base sintética de colaboradores na primeira execução. O passo a passo detalhado fica na wiki, e não neste documento, porque é o tipo de informação que muda com frequência.

A garantia não para no ambiente local: como a esteira de integração contínua e os ambientes de homologação e produção executam essas mesmas imagens, um comportamento observado na máquina de um integrante é reproduzível na verificação automatizada e no servidor que atende o cliente. O clássico *funciona na minha máquina* deixa de ser um argumento possível, porque a máquina é a mesma em todos os pontos do processo.

A mesma base de código é construída de duas formas diferentes, conforme o objetivo. Não são dois softwares, e sim duas configurações de construção:

- **Desenvolvimento:** prioriza o tempo de retorno. O código não é minificado, os *source maps* são preservados, o servidor recarrega automaticamente a cada alteração, o nível de registro é detalhado e a base é populada com dados sintéticos.
- **Produção:** prioriza desempenho e segurança. O *frontend* é compilado e minificado em arquivos estáticos, o TypeScript é transpilado antecipadamente, as dependências de desenvolvimento ficam fora da imagem final, o registro detalhado é desligado para não expor dados de colaboradores e as migrações são aplicadas de forma controlada antes de a nova versão entrar no ar.

Em nenhum dos casos há configuração específica de ambiente escrita no código: o que muda são variáveis de ambiente, de modo que a mesma imagem verificada pela automação é a que roda para o cliente.

6.4.2 Integração contínua

A verificação automatizada roda no GitHub Actions e é acionada em duas situações: a cada *push* em um *branch* de trabalho e a cada *pull request* aberto ou atualizado para a *main*. A Tabela 6.3 descreve os trabalhos que compõem a esteira.

A *main* exige que todas essas verificações estejam aprovadas e que haja ao menos uma aprovação humana antes da integração, permanecendo sempre em estado construível: qualquer

Tabela 6.3: Verificações automatizadas executadas pela integração contínua.

Verificação	Quando executa	O que valida
Padrão de <i>commit</i>	<i>Pull request</i>	A mensagem segue o padrão adotado e contém a referência <code>TG-<ref></code> , sem a qual a rastreabilidade da Seção 6.3 se perde.
Análise estática	<i>Push</i> e <i>pull request</i>	ESLint e Prettier nos projetos TypeScript, Ruff nos serviços Python. Falha em erro de formatação ou de estilo, o que mantém a parte mecânica do padrão de código fora da discussão na revisão.
Compilação	<i>Push</i> e <i>pull request</i>	O TypeScript compila sem erros de tipo e as imagens de contêiner são construídas com sucesso.
Testes de unidade	<i>Push</i> e <i>pull request</i>	Executa a suíte de unidade e publica o relatório de cobertura como comentário no <i>pull request</i> .
Testes de integração	<i>Pull request</i> para a <code>main</code>	Sobe um PostgreSQL como serviço da esteira e executa os testes que atravessam API, repositórios e banco.
Documentação	Alterações em <code>projeto/</code>	Compila este documento com <code>latexmk</code> e falha se houver erro de <code>L^AT_EX</code> ou referência indefinida.

integrante pode partir dela para uma nova tarefa sem antes precisar descobrir se está quebrada, o que importa em uma equipe que trabalha de forma assíncrona e em horários distintos.

Vale destacar a divisão de trabalho entre a esteira e a revisão humana: a análise estática cobre a parte mecânica do padrão de código — formatação, nomenclatura e regras de estilo —, que por isso não precisa mais ser discutida nos comentários de revisão. O que a ferramenta não consegue julgar, como se a abstração criada é a certa, se o comentário explica o que precisava ser explicado e se a alteração respeita a organização de módulos descrita na wiki, fica com o revisor, o que mantém a revisão de código focada em decisões de projeto.

6.4.3 Ambientes e entrega

O software passa por três ambientes até chegar ao usuário. O de **desenvolvimento** é a máquina de cada integrante, levantada pelo `docker-compose`. O de **homologação** é uma réplica do ambiente de produção, com a mesma configuração de construção e uma base de dados sintética; toda integração na `main` é implantada nele automaticamente, e é ali que a equipe e o responsável pelo produto validam o incremento antes da liberação. O de **produção** atende os usuários da organização cliente.

A entrega para produção é acionada pela criação de uma *tag* de versão, ao final da *sprint* e após o aceite em homologação. O fluxo de *release* constrói as imagens em modo de produção, executa a suíte completa de testes, publica uma *release* no GitHub com as notas da versão e o PDF desta documentação, aplica as migrações pendentes e sobe a nova versão.

O objetivo de confiabilidade definido na Seção 1.5 tem consequência direta sobre esse último passo: como o sistema precisa permanecer disponível durante o horário comercial, a subida é feita de forma gradual, com as instâncias sendo substituídas uma a uma, só sendo considerada concluída depois que a nova versão responde corretamente à verificação de saúde. Migrações são escritas de modo a manter compatibilidade com a versão anterior durante a transição, o que permite reverter para a *tag* anterior sem perder dados caso algum problema apareça logo após

a liberação, e a implantação em si depende de uma aprovação explícita na esteira: a equipe automatizou a execução, mas manteve a decisão de liberar como um ato humano e consciente.

O que essa automação entrega, no fim, é confiança para entregar com frequência: a equipe consegue publicar uma versão a cada duas semanas sabendo que o que está na `main` compila, passa nos testes, mantém o vínculo com o item de trabalho que o originou e pode ser revertido em minutos. Sem isso, o custo de cada entrega cresce, a frequência cai e o software passa a mudar mais devagar do que a organização que depende dele.

Controle de Versões do Documento

Orientação: Registrem apenas alterações significativas nesta documentação. Este controle se refere ao documento; o versionamento do software deve ser descrito no capítulo de Gestão de Configuração e Manutenção.

Tabela 6.4: Histórico de versões do documento.

Data	Autor(es)	Versão	Alteração
DD/MM/AAAA	Equipe	1.0	Versão inicial para entrega.

Referências bibliográficas

International Organization for Standardization. *ISO/IEC 25010:2011 – Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models*. ISO, 2011.